CPSC 321: Homework 4 (Queries 2) *Due: Thursday, Oct. 15, 2026*

Instructions. This homework uses your factbook tables from HW-3 and the movie rental database on the course server. Submit these three files to Canvas by 11:59 PM on Thursday, October 15:

- `hw4_factbook.sql`: table setup, data, and answers to Questions 1–8 (Parts 1 and 2).
- `hw4_movies.sql`: answers to Questions 9–12 (Part 3).
- `hw4.pdf`: results and explanations, organized by question number (Part 4).

SQL Scripts. Use the SQL header format from HW-3, updated for this assignment and each script. Use a consistent SQL style based on the style guides linked from the course webpage. Label each answer with a comment giving its question number, purpose, and any chosen parameter values or assumptions.

Allowed SQL. Use only the constructs listed below.

- Everything allowed in HW-3.
- Outer joins (`LEFT`, `RIGHT`, and `FULL`), `IS NULL`, `IS NOT NULL`, and `COALESCE`.
- The set operations `UNION`, `UNION ALL`, `INTERSECT`, and `EXCEPT`.
- `CREATE VIEW`, `PREPARE` and `EXECUTE`, and `CREATE FUNCTION` and `CREATE PROCEDURE` with `LANGUAGE SQL`, along with the corresponding `DROP` statements, `DEALLOCATE`, and `CALL`.
- Date and time arithmetic as described in the Data Definition reader (e.g., `INTERVAL '7 days'`).

Avoid unnecessary joins, and use `DISTINCT` and `ORDER BY` only when needed. Use `JOIN` syntax for all joins (no comma joins). You *may not* use subqueries, aggregate functions, `GROUP BY`, `HAVING`, `LIMIT`, `CASE`, `WITH` clauses, or PL/pgSQL.

PART 1. Copy your `hw3_1.sql` script to `hw4_factbook.sql`. The schema, including constraints and required (`NOT NULL`) attributes, is the same as in HW-3:

- `country(country_code, country_name, gdp, inflation)`
- `province(province_name, country_code, area)`
- `city(city_name, province_name, country_code, population)`
- `border(country_code_1, country_code_2, border_length)`

Add these "cleanup" commands before your `DROP TABLE` statements so that you can rerun the completed script:

```
DROP VIEW IF EXISTS border_full;
DROP FUNCTION IF EXISTS large_cities;
DROP PROCEDURE IF EXISTS add_province;
DEALLOCATE ALL;
```

Extend your data to include:

- A country with no borders.
- At least two countries tied for the highest GDP per capita.
- A country with a lower GDP per capita.

Also include the data needed for the examples requested with each question below.

PART 2. Add your answers to Questions 1–8 below to `hw4_factbook.sql`, after the table definitions and starting data from Part 1. Be sure to run the entire `hw4_factbook.sql` script each time you test. It must restore the starting data and execute your answers without errors.

1. Create a view named `border_full` with the columns `country_code_1`, `country_code_2`, and `border_length`. The view must contain each border in both directions. That is, if (x, y, ℓ) is a row in `border`, then `border_full` must contain both (x, y, ℓ) and (y, x, ℓ). Your view definition must use a set operation. After the view definition, add `SELECT * FROM border_full;` to your script.

2. Rewrite your query from HW-3 Question 8 using the `border_full` view. You must return the country code and country name, with only one row per matching country. Use the same threshold as in HW-3. Your query must not use `OR` and must return at least two rows.

   In your write-up: Run your HW-3 Question 8 query on the same starting data and confirm that the results match.

3. Write a query using `EXCEPT` to find the countries that do not border any other country. Return the country code and country name. You may use the `border_full` view.

   In your write-up: Give one matching and one nonmatching example.

4. Rewrite your query from Question 3 using an outer join instead of `EXCEPT`. You may use the `border_full` view.

   In your write-up: Confirm that the output matches Question 3.

5. Write a query to find the countries with the highest GDP per capita among all countries. Return the country code, country name, and GDP per capita. If several countries tie for the highest GDP, return all of them. *Hint*: Consider using a set operation.

   In your write-up: Give one matching and one nonmatching example.

6. Write a prepared statement named `country_cities` that takes a country code and a minimum population, and returns the city name, province name, and population of each city in that country whose population is at least the minimum. Then use `EXECUTE` to run it twice, with two different countries and minimum populations. Each execution must return at least two rows. For your first execution, include two cities in the chosen country in your Part 1 starting data: one with a population equal to your chosen minimum and one below it.

7. Write a stored function named `large_cities` that takes the same two parameters and returns the same columns as Question 6. Then write a query that calls the function with the same country and minimum population as your first `EXECUTE` statement in Question 6. For each city, return the city name, population, province name, and the area of its province. Sort the result from largest to smallest population.

   In your write-up: Confirm that the city names, province names, and populations in your query's output match the first execution in Question 6.

8. Write a stored procedure named `add_province` that takes a country code, a province name, a province area, a city name, and a city population. The procedure adds the province, and

then adds the city to that province. After the procedure, add a `CALL` statement that uses it to add a new province and city to your data. After your `CALL`, include a query that displays the province and city you added.

PART 3. Place your answers to Questions 9–12 below in `hw4_movies.sql`. Use the movie rental database `cpsc321_db` on the course server. From `psql`, connect using `\c cpsc321_db`. The relevant tables and columns are listed below. Use `\d` *table* to see a table's full definition.

- `customer(customer_id, store_id, first_name, last_name, email, ...)`
- `rental(rental_id, rental_date, inventory_id, customer_id, return_date, ...)`
- `inventory(inventory_id, film_id, store_id, ...)`
- `film(film_id, title, length, rating, ...)`
- `actor(actor_id, first_name, last_name, ...)`
- `film_actor(actor_id, film_id, ...)`
- `category(category_id, name, ...)`
- `film_category(film_id, category_id, ...)`

Each row of `inventory` is one copy of a film at one of the two stores. Each row of `rental` is one rental of one copy. A `NULL` `return_date` means the copy has not been returned yet. The `length` of a film is in minutes. Unlike in Part 2, the data is fixed, so use the exact values given in each question.

9. Find the rentals of G-rated films from store 1 that have not been returned yet. Use the store of the rented copy (in `inventory`). Return the customer's first name, last name, and email, and the film title and rental date. Sort the result by the customer's last name and then first name.

10. Find the rentals by customer 43 that were returned more than 7 days after the rental date, or that have not been returned yet. Return the film title, rental date, and return date. Sort the result by rental date.

11. Find the G-rated films that have no copies in the inventory of either store. Use an outer join. Return the film ID and title, sorted by title.

12. Find the actors who appeared in a Horror film that is at least 170 minutes long and also appeared in a Sci-Fi film that is at least 170 minutes long. You may use `INTERSECT` in your query. Return the actor ID, first name, and last name. Return each actor only once. Sort the result by last name and then first name.

PART 4. Organize your typed `hw4.pdf` write-up by question number (and include the question numbers in your answers). For each question, include the output of all queries and `EXECUTE` statements. Include any chosen parameter values and the explanations requested with each question. Copy output from PostgreSQL using your final scripts, with a monospaced font that preserves column headings and row alignment. Where matching and nonmatching examples are requested, show the relevant stored rows and briefly explain why each example qualifies or does not qualify.